used in networking :
- routing (popcnt IP lookup)
- netmask / host-bit calc
- subnet mask (ACL / firewall)
- cache bloomfilters

$\sum_{k=0}^{63} (rcx >> k)$

ie popcnt rax

rax == popcount(rax)

ie # set bits ∈ rax

```
    xor     eax,eax     eax:=0
.loop:
    jrcxz   .exit_loop
    inc     rax
    mov     rdx,rcx
    dec     rdx
    and     rcx,rdx
    jmp     .loop
.exit_loop:
```

while (rcx != 0)
{
$rax_{n+1} = rax_n + 1$
$rdx_n = rcx_i - 1$
$rcx = rcx \wedge rdx$
$rcx = rcx_i \wedge (rcx_i - 1)$] → removes lowest 2^k term from binary value of rcx

ie removes rightmost-bit

ex 1001000 11

100100010

100100002

100000000

0

```
xor     eax, eax        eax := 0

.loop:
    jrcxz   .exit_loop      "jump shore
                             if rcx == 0"
    inc     rax             rax++  → count loop
    mov     rdx, rcx        rdx := rcx ≠ 0
    dec     rdx             rdx = rcx - 1
    and     rcx, rdx
    jmp     .loop

.exit_loop:
```

"odd - 1, even - 2, 8, 0 → 0"

ie use 'one' or 'two' of rcx until it is 0, & record how long this takes w/ rax

rounding up!

$\left(\frac{n}{2}\right) = rax$

9.5

$4 - 2 = 2 - 2 = 0$

$17 - 1 = 16 - 2 = 14 - 2 = 12 - 2 = 10 - 2 = 8 - 2 = 6 - 2 = 4$

$16 \to 8 \cdot 5$